

YEAR 2 – DATA STRUCTURES & ALGORITHMS

PROJECT REPORT

RouteForge

A Graph Algorithm Visualisation System

Module: Data Structures & Algorithms

Language: Python 3.11+

Framework: pygame 2.x

Authors

Blevis Allushi (AI)

Kristian Seraj (AI)

Renato Zotaj (SWE)

Leandra Latifi (AI)

Contents

1	Introduction	2
2	System Architecture	2
2.1	Module Layout	2
2.2	Module Dependencies	3
2.3	Validation Architecture	3
3	Core Data Structures	4
3.1	Graph Representation	4
3.2	Edge Dataclass	4
3.3	Disjoint Set (Union-Find)	5
4	Input Validation	5
5	Algorithms	6
5.1	Breadth-First Search	6
5.2	Depth-First Search	7
5.3	Dijkstra’s Shortest-Path Algorithm	8
5.4	Minimum Spanning Tree Algorithms	9
5.4.1	Prim’s Algorithm	9
5.4.2	Kruskal’s Algorithm	10
5.4.3	Comparison	10
6	Graph Visualiser	10
6.1	Overview	10
6.2	Interaction State Machine	11
6.3	Animation Model	11
6.4	Visual Encoding	12
6.5	Playback Controls	12
6.6	Edit Mode Controls	13
7	Random Graph Generator	13
8	Command-Line Interface	14
9	Conclusion	15
9.1	Potential Extensions	15

1 Introduction

Graph theory underpins a broad class of real-world problems – from network routing and geographic pathfinding to social-network analysis and dependency resolution. Understanding how fundamental graph algorithms operate is a core competency in computer science education; yet the abstract mechanics of queues, priority queues, and predecessor maps are often difficult to grasp from code or pseudocode alone.

RouteForge addresses this gap by pairing correct, modular algorithm implementations with an interactive step-by-step visualiser. The system is designed around three principles:

- (i) **Separation of concerns** – the graph data structure, the algorithms, the CLI, and the visualiser are entirely independent modules.
- (ii) **Correctness first** – each algorithm is implemented faithfully to its canonical form; the visualiser replicates the algorithm logic internally without modifying the production code.
- (iii) **Interactivity** – the user can construct arbitrary graphs, generate random instances, and animate algorithms at configurable speeds, stepping forwards and backwards through execution.

2 System Architecture

2.1 Module Layout

The project is organised as a Python package rooted at `src/`.

```
RouteForge/  
|-- main.py  
'-- src/  
    |-- algorithms/  
    |   |-- bfs.py  
    |   |-- dfs.py  
    |   |-- dijkstra.py  
    |   |-- helpers.py  
    |   |-- kruskal.py  
    |   '-- prim.py  
    |-- core/  
    |   |-- graph.py  
    |   '-- validators.py  
    |-- data/  
    |-- ui/
```

```

|   '-- menu.py
|-- utils/
|   '-- graph_generator.py
'-- visualization/
    '-- pygame_view.py

```

2.2 Module Dependencies

The dependency structure is intentionally acyclic and layered:

- `src/core/` has no internal dependencies; it defines the foundation types used by every other layer.
- `src/algorithms/` depends only on `src/core/`.
- `src/utils/` depends only on `src/core/`.
- `src/visualization/` depends on `src/core/` only.
- `src/ui/` depends on all other layers and acts as the integration point.
- `main.py` depends only on `src/ui/menu.py`.

This layering ensures that the core graph and algorithm modules can be tested and used independently of the interface layers.

2.3 Validation Architecture

Input validation is split across three layers:

Table 1: Validation responsibility by layer.

Layer	Module	Responsibility
Domain rules	<code>validators.py</code>	Node name format, weight range, NaN/Inf rejection, self-loop prevention
Structural	<code>graph.py</code>	Node existence, endpoint auto-creation, adjacency consistency
UI / interaction	<code>pygame_view.py</code>	Scroll-wheel guard, click blocking during input, error message display

3 Core Data Structures

3.1 Graph Representation

RouteForge models a weighted undirected graph $G = (V, E, w)$ using an *adjacency list* stored as a nested Python dictionary:

$$_adj : \text{str} \rightarrow (\text{str} \rightarrow \text{float})$$

Each outer key is a node name; its value maps neighbour names to edge weights. Every undirected edge $\{u, v, w\}$ is stored as two directed entries so that $_adj[u][v] = _adj[v][u] = w$.

The adjacency list was chosen over an adjacency matrix because it uses $O(|V| + |E|)$ space, supports $O(1)$ edge lookup via nested dict membership, and accommodates dynamic node/edge addition without reallocation.

Table 2: Time complexity of core **Graph** operations.

Operation	Complexity	Notes
<code>add_node</code>	$O(1)$	Dict insert
<code>add_edge</code>	$O(1)$	Two dict inserts + validation
<code>remove_node</code>	$O(\deg(v))$	Removes reverse entries
<code>remove_edge</code>	$O(1)$	Two dict deletes
<code>has_node</code>	$O(1)$	Dict membership
<code>has_edge</code>	$O(1)$	Nested dict membership
<code>neighbors</code>	$O(\deg(v))$	Returns list of (neighbour, weight)
<code>edges</code>	$O(V + E)$	Deduplication via canonical key
Space	$O(V + E)$	

3.2 Edge Dataclass

Edges are returned as instances of a frozen, hashable **Edge** dataclass, enabling their use in sets and as dictionary keys:

```

1 @dataclass(frozen=True)
2 class Edge:
3     u: str
4     v: str
5     weight: float

```

Listing 1: Edge dataclass (`src/core/graph.py`).

The `edges()` method deduplicates using a canonical key ($\min(u, v), \max(u, v)$), ensuring each undirected edge appears exactly once regardless of insertion order.

3.3 Disjoint Set (Union-Find)

Kruskal’s algorithm requires a disjoint-set structure to detect cycles. The `DisjointSet` class implements union by rank with path compression, giving an amortised cost of $O(\alpha(n))$ per operation where α is the inverse Ackermann function – effectively constant for all practical inputs.

```

1 class DisjointSet:
2     def __init__(self, items):
3         self.parent = {item: item for item in items}
4         self.rank   = {item: 0   for item in items}
5
6     def find(self, x):
7         if self.parent[x] != x:
8             self.parent[x] = self.find(self.parent[x]) # path
9                 compression
10
11         return self.parent[x]
12
13     def union(self, a, b):
14         root_a, root_b = self.find(a), self.find(b)
15         if root_a == root_b:
16             return False
17         if self.rank[root_a] < self.rank[root_b]:
18             self.parent[root_a] = root_b
19         elif self.rank[root_a] > self.rank[root_b]:
20             self.parent[root_b] = root_a
21         else:
22             self.parent[root_b] = root_a
23             self.rank[root_a] += 1
24         return True

```

Listing 2: Disjoint-Set (`src/algorithms/kruskal.py`).

4 Input Validation

All domain-level rules live in `validators.py` and are called by `add_node` and `add_edge` before any structural change occurs. The UI layer also calls validators independently to surface precise error messages in the status bar without crashing.

- **Node names** – non-empty, at most 16 characters, restricted to `[a-zA-Z0-9_-]`.
- **Weights** – numeric, finite (no NaN or $\pm\infty$), within $[0.0, 10^6]$.
- **Edges** – both endpoint names must be valid and distinct (no self-loops).

All validators raise `ValueError` with a descriptive message on failure.

5 Algorithms

5.1 Breadth-First Search

BFS explores a graph level by level using a FIFO queue, visiting all neighbours of a node before advancing to their neighbours. Neighbours are processed in lexicographic order to ensure a deterministic traversal.

Algorithm 1 Breadth-First Search

```

1: function BFS( $G$ ,  $start$ )
2:    $visited \leftarrow \{start\}$ ;  $queue \leftarrow \langle start \rangle$ ;  $order \leftarrow []$ 
3:   while  $queue \neq \emptyset$  do
4:      $node \leftarrow \text{DEQUEUE}(queue)$ 
5:      $order.APPEND(node)$ 
6:     for each  $(nbr, \_)$  in  $\text{SORTEDNEIGHBOURS}(G, node)$  do
7:       if  $nbr \notin visited$  then
8:          $visited.ADD(nbr)$ ;  $\text{ENQUEUE}(queue, nbr)$ 
9:       end if
10:    end for
11:  end while return  $order$ 
12: end function

```

`collections.deque` is used for $O(1)$ `popleft` operations; a plain list would give $O(n)$ `pop(0)`.

```

1 def bfs(graph: Graph, start: str) -> List[str]:
2     if not graph.has_node(start):
3         raise KeyError(f"Start node '{start}' does not exist.")
4     visited = set([start])
5     queue = deque([start])
6     order: List[str] = []
7     while queue:
8         node = queue.popleft()
9         order.append(node)
10        for neighbor, _ in sorted(graph.neighbors(node), key=lambda
11            x: x[0]):
12            if neighbor not in visited:
13                visited.add(neighbor)
14                queue.append(neighbor)
15    return order

```

Listing 3: BFS implementation (src/algorithms/bfs.py).

Complexity: $O(|V| + |E|)$ time, $O(|V|)$ space.

5.2 Depth-First Search

DFS explores as far as possible along each branch before backtracking. An iterative implementation with an explicit stack is used to avoid Python's recursion depth limit. Neighbours are pushed in reverse-sorted order so the lexicographically smallest is expanded first, matching recursive DFS behaviour. The visited check occurs on *pop* rather than *push* because a node may be pushed multiple times before first expansion.

Algorithm 2 Depth-First Search (Iterative)

```

1: function DFS( $G, start$ )
2:    $visited \leftarrow \emptyset$ ;  $stack \leftarrow [start]$ ;  $order \leftarrow []$ 
3:   while  $stack \neq \emptyset$  do
4:      $node \leftarrow stack.POP()$ 
5:     if  $node \in visited$  then continue
6:   end if
7:    $visited.ADD(node)$ ;  $order.APPEND(node)$ 
8:   for each  $(nbr, \_)$  in  $REVERSESORTEDNEIGHBOURS(G, node)$  do
9:     if  $nbr \notin visited$  then
10:       $stack.PUSH(nbr)$ 
11:    end if
12:  end for
13: end while return  $order$ 
14: end function

```

```

1 def dfs(graph: Graph, start: str) -> List[str]:
2     if not graph.has_node(start):
3         raise KeyError(f"Start node '{start}' does not exist.")
4     visited = set()
5     stack = [start]
6     order: List[str] = []
7     while stack:
8         node = stack.pop()
9         if node in visited:
10             continue
11         visited.add(node)
12         order.append(node)
13         neighbors = sorted(graph.neighbors(node), key=lambda x: x[0],
14                             reverse=True)
15         for neighbor, _ in neighbors:
16             if neighbor not in visited:
17                 stack.append(neighbor)

```

```
17     return order
```

Listing 4: DFS implementation (`src/algorithms/dfs.py`).

Complexity: $O(|V| + |E|)$ time, $O(|V|)$ space.

5.3 Dijkstra's Shortest-Path Algorithm

Dijkstra's algorithm computes single-source shortest paths in graphs with non-negative edge weights. It greedily expands the nearest unvisited node using a min-priority queue and relaxes outgoing edges. The `validate_weight` call in `add_edge` enforces $w \geq 0$ at construction time, guaranteeing the correctness precondition.

Algorithm 3 Dijkstra's Algorithm

```

1: function DIJKSTRA( $G, start$ )
2:    $dist[v] \leftarrow \infty$  for all  $v$ ;  $dist[start] \leftarrow 0$ ;  $prev[v] \leftarrow \text{NONE}$ 
3:    $pq \leftarrow \{(0, start)\}$ ;  $visited \leftarrow \emptyset$ 
4:   while  $pq \neq \emptyset$  do
5:      $(d, node) \leftarrow \text{EXTRACTMIN}(pq)$ 
6:     if  $node \in visited$  then continue
7:     end if
8:      $visited.ADD(node)$ 
9:     for each  $(nbr, w)$  in  $\text{NEIGHBOURS}(G, node)$  do
10:      if  $d + w < dist[nbr]$  then
11:         $dist[nbr] \leftarrow d + w$ ;  $prev[nbr] \leftarrow node$ 
12:         $pq.INSERT(d + w, nbr)$ 
13:      end if
14:    end for
15:   end while return  $dist, prev$ 
16: end function
```

Lazy deletion is used: stale heap entries are discarded by the *visited* check on extraction, avoiding the need for a decrease-key operation.

```

1 def dijkstra(graph, start):
2     distances = {node: float("inf") for node in graph.nodes()}
3     previous = {node: None for node in graph.nodes()}
4     distances[start] = 0.0
5     pq = [(0.0, start)]
6     visited = set()
7     while pq:
8         current_dist, node = heapq.heappop(pq)
9         if node in visited:
```

```

10         continue
11     visited.add(node)
12     for neighbor, weight in graph.neighbors(node):
13         new_dist = current_dist + weight
14         if new_dist < distances[neighbor]:
15             distances[neighbor] = new_dist
16             previous[neighbor] = node
17             heapq.heappush(pq, (new_dist, neighbor))
18     return distances, previous

```

Listing 5: Dijkstra core (src/algorithms/dijkstra.py).

Path reconstruction is handled by `reconstruct_path` in `helpers.py`, which walks the predecessor map backwards from the end node and reverses the result. If the end node is unreachable, `None` is returned.

Complexity: $O((|V| + |E|) \log |V|)$ time, $O(|V|)$ space.

5.4 Minimum Spanning Tree Algorithms

Both MST algorithms are exposed through the CLI and visualised step-by-step in the pygame interface.

5.4.1 Prim's Algorithm

Prim's algorithm grows the MST one edge at a time, repeatedly selecting the minimum-weight edge crossing the frontier between visited and unvisited nodes. It requires a connected graph and an optional start node (defaults to the first node).

Algorithm 4 Prim's MST

```

1: function PRIMMST( $G, start$ )
2:    $visited \leftarrow \{start\}$ ;  $mst \leftarrow []$ ; push all edges from  $start$  into  $pq$ 
3:   while  $pq \neq \emptyset$  and  $|visited| < |V|$  do
4:      $(w, u, v) \leftarrow \text{EXTRACTMIN}(pq)$ 
5:     if  $v \in visited$  then continue
6:     end if
7:      $visited.ADD(v)$ ;  $mst.APPEND(EDGE(u, v, w))$ 
8:     for each  $(nbr, wt)$  in  $NEIGHBOURS(G, v)$  where  $nbr \notin visited$  do
9:        $pq.INSERT(wt, v, nbr)$ 
10:    end for
11:  end while return  $mst$ 
12: end function

```

Complexity: $O(|E| \log |V|)$ time, $O(|V| + |E|)$ space.

5.4.2 Kruskal's Algorithm

Kruskal's algorithm sorts all edges by weight and adds each edge to the MST if its endpoints belong to different components, using the `DisjointSet` structure for cycle detection. The loop exits early once $|V| - 1$ edges have been accepted.

Algorithm 5 Kruskal's MST

```

1: function KRUSKALMST( $G$ )
2:    $ds \leftarrow \text{DISJOINTSET}(V)$ ;  $edges \leftarrow \text{SORTEDBYWEIGHT}(E)$ 
3:    $mst \leftarrow []$ 
4:   for each  $e = (u, v, w)$  in  $edges$  do
5:     if  $\text{UNION}(ds, u, v)$  then
6:        $mst.\text{APPEND}(e)$ 
7:       if  $|mst| = |V| - 1$  then break
8:     end if
9:   end if
10:  end for return  $mst$ 
11: end function

```

Complexity: $O(|E| \log |E|)$ time (dominated by sorting), $O(|V|)$ space.

5.4.3 Comparison

Table 3: Prim vs. Kruskal.

Property	Prim	Kruskal
Approach	Vertex-centric; grows one component	Edge-centric; merges components
Data structure	Min-heap	Disjoint-set + sorted edge list
Preferred for	Dense graphs	Sparse graphs
Start node	Required	Not required

6 Graph Visualiser

6.1 Overview

The visualiser (`src/visualization/pygame_view.py`) is a self-contained pygame application supporting two primary modes:

Edit mode The user constructs a graph interactively: left-clicking empty space creates a node; clicking two nodes in sequence opens a floating weight-input box to create an edge.

Algorithm mode BFS, DFS, Dijkstra, Prim, or Kruskal is animated step-by-step with full playback controls.

6.2 Interaction State Machine

The visualiser tracks a string-valued mode with four states:

Table 4: Visualiser mode states.

State	Trigger	Description
"edit"	Initial / R	Graph editing enabled
"pick_start"	B, D, K, P, U	Awaiting start node
"pick_end"	Start chosen (Dijkstra only)	Awaiting end node
"algo"	Node(s) confirmed	AlgoPlayer active

All states return to "edit" via Escape or R.

6.3 Animation Model

The step generators (`_bfs_steps`, `_dfs_steps`, `_dijkstra_steps`, `_prim_steps`, `_kruskal_steps`) replicate each algorithm's logic exactly, recording an `AlgoStep` snapshot after every meaningful event. All steps are precomputed at launch, enabling instant backward stepping and variable-speed playback without re-running the algorithm.

```

1 @dataclass
2 class AlgoStep:
3     visited      : Set[str]
4     frontier     : Set[str]
5     active_node  : Optional[str]
6     active_edges : Set[Tuple[str, str]]
7     active_edge  : Optional[Tuple[str, str]]
8     dist_labels  : Dict[str, str]           # Dijkstra only
9     path_nodes   : Set[str]                 # Dijkstra final step
10    path_edges   : Set[Tuple[str, str]]      # Dijkstra / MST final step
11    mst_edges     : Set[Tuple[str, str]]      # Prim / Kruskal
12    description   : str

```

Listing 6: `AlgoStep` dataclass.

6.4 Visual Encoding

Table 5: Node and edge colour encoding during algorithm playback.

Element	Colour	Meaning
Node – default	Blue (30, 120, 220)	Unvisited
Node – frontier	Orange (220, 130, 30)	In queue / stack / heap
Node – active	White (240, 240, 255)	Being expanded this step
Node – visited	Teal (20, 160, 140)	Fully processed
Node – path	Gold (255, 180, 20)	On Dijkstra shortest path
Node – MST	Purple	In MST (Prim / Kruskal)
Edge – default	Grey (140, 155, 175)	Untraversed
Edge – traversed	Teal (20, 160, 140)	Crossed during traversal
Edge – active	White	Being relaxed / added now
Edge – path/MST	Gold (255, 210, 50)	Final shortest path or MST edge

For Dijkstra, the current best-known distance from the source is rendered above each node and updated at every step.

6.5 Playback Controls

Table 6: Keyboard controls during playback.

Key	Action
Space	Play / Pause
Right arrow	Step forward
Left arrow	Step backward
+ / =	Increase speed
-	Decrease speed
R / Esc	Reset to edit mode

Speed levels cycle through $\{0.5, 1, 2, 4, 8\}$ steps per second. The `AlgoPlayer` time accumulator advances the step index at the correct rate regardless of frame rate.

6.6 Edit Mode Controls

Table 7: Edit mode input controls.

Input	Action
Left-click empty space	Add node
Left-click node	Select as edge start
Left-click second node	Open weight input
Left-click same node	Deselect
Right-click node (no selection)	Delete node and all incident edges
Right-click edge	Delete edge
Right-click (mid-selection)	Cancel current selection
Scroll wheel	Ignored
Enter	Confirm edge weight
Backspace	Delete character in weight input
Escape	Cancel action
B	Launch BFS
D	Launch DFS
K	Launch Dijkstra
P	Launch Prim's MST
U	Launch Kruskal's MST

7 Random Graph Generator

`generate_random_graph` in `src/utils/graph_generator.py` produces a random connected weighted graph in two phases:

1. **Connectivity chain** – a Hamiltonian path $N_0 - N_1 - \dots - N_{n-1}$ with random weights in $[1, \text{max_weight}]$ guarantees connectivity.
2. **Random augmentation** – for each pair (i, j) with $i < j$, an additional edge is added with probability `edge_probability` (default 0.4).

```

1 def generate_random_graph(n_nodes=6, edge_probability=0.4,
   max_weight=10):
2     g = Graph()
3     nodes = [f"N{i}" for i in range(n_nodes)]
4     for n in nodes:
5         g.add_node(n)
6     for i in range(n_nodes - 1):                # phase 1: chain
7         g.add_edge(nodes[i], nodes[i + 1], random.randint(1,
           max_weight))
8     for i in range(n_nodes):                    # phase 2: augment

```

```

9         for j in range(i + 1, n_nodes):
10             if random.random() < edge_probability:
11                 g.add_edge(nodes[i], nodes[j], random.randint(1,
12                     max_weight))
12     return g

```

Listing 7: Random graph generator (`src/utils/graph_generator.py`).

The expected edge count after augmentation is $(n - 1) + \binom{n}{2} \cdot p$. For the defaults $n = 6$, $p = 0.4$ this gives approximately 11 edges. Phase 1 edges may be overwritten by Phase 2 if the same pair is selected, which is harmless since `add_edge` updates the weight in place.

8 Command-Line Interface

The `Menu` class in `src/ui/menu.py` provides a numbered text menu that exposes the full graph API and all five algorithms:

Table 8: CLI menu options.

Option	Action
1	Add node
2	Add edge
3	Display adjacency list
4	BFS traversal
5	DFS traversal
6	Shortest path (Dijkstra)
7	Minimum spanning tree (Prim)
8	Minimum spanning tree (Kruskal)
9	Generate random graph
10	Open pygame visualiser
11	Remove node
12	Remove edge
13	Load sample graph
0	Exit

The graph object persists across menu invocations. When option 10 is selected, the current `Graph` instance is passed directly to `GraphVisualizer`, so any graph built or loaded via the CLI is immediately available for visual inspection.

9 Conclusion

RouteForge delivers a modular, correct, and interactive graph algorithm system. The separation of core data structures, algorithm implementations, validation, and visualisation into independent modules means each component can be tested, extended, or replaced in isolation.

The precomputed step-list animation model supports instant backward stepping and variable-speed playback – capabilities that live execution would not allow. The consistent visual encoding across all five algorithms (BFS, DFS, Dijkstra, Prim, Kruskal) lets users compare how each algorithm explores the same graph under identical starting conditions.

9.1 Potential Extensions

1. **Directed graph support** – extend `Graph` for directed edges, enabling topological sort and Bellman-Ford.
2. **A* search** – a natural extension for geographic pathfinding with heuristic guidance.
3. **Graph import/export** – JSON or GraphML serialisation for persistent graphs and external tool interoperability.
4. **Drag-and-drop node repositioning** – mouse drag support for improved layout control on larger graphs.
5. **Unit test suite** – the `tests/` directory is currently empty; property-based tests using `hypothesis` would significantly increase correctness confidence.